

Experiment No. 27

Title

Prolog Program to Implement Best First Search Algorithm

Aim

To write a Prolog program to implement the **Best First Search algorithm**.

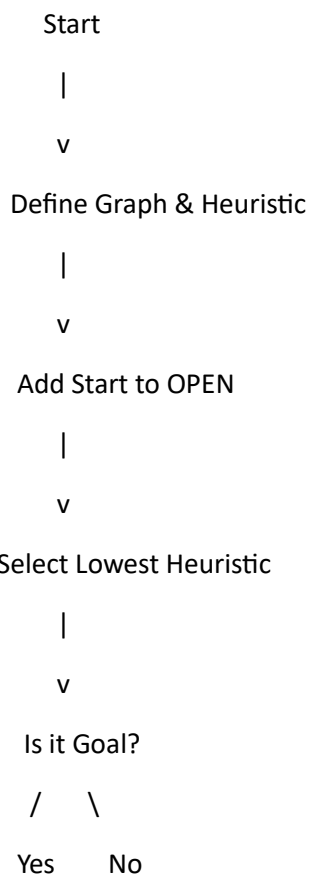
Objective

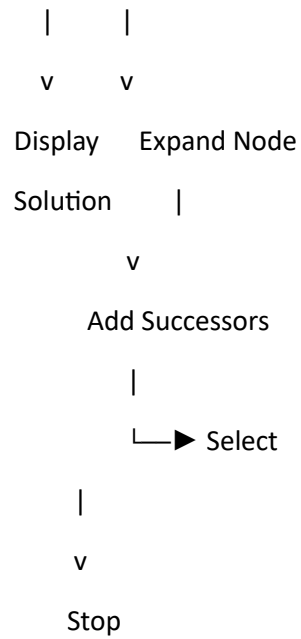
To understand how heuristic values are used to select the most promising node during search.

Algorithm

1. Start.
2. Define the graph and heuristic values.
3. Add the initial node to the open list.
4. Select the node with the lowest heuristic value.
5. Expand the selected node.
6. Repeat until the goal is reached.
7. Display the solution path.

Flowchart





Prolog Program

edge(a,b).

edge(a,c).

edge(b,d).

edge(b,e).

edge(c,f).

edge(c,g).

heuristic(a,6).

heuristic(b,4).

heuristic(c,3).

heuristic(d,2).

heuristic(e,5).

heuristic(f,1).

heuristic(g,0).

best_first(Start, Goal, Path) :-

 search([Start], Goal, [], Path).

search([Goal|_], Goal, _, [Goal]).

```
search([Current|Rest], Goal, Visited, [Current|Path]) :-
```

```
    findall(H-Next,  
            (edge(Current, Next),  
              \+ member(Next, Visited),  
              heuristic(Next, H)),  
            Children),  
    keysort(Children, Sorted),  
    pairs(Sorted, Nodes),  
    append(Nodes, Rest, NewQueue),  
    search(NewQueue, Goal, [Current|Visited], Path).
```

```
pairs([], []).
```

```
pairs([_X|T], [X|R]) :-
```

```
    pairs(T, R).
```

Query

```
?- best_first(a,g,Path).
```

Sample Output

```
Path = [a,c,g].
```

Result

The Prolog program successfully implemented **Best First Search** and found a path from the starting node to the goal using heuristic values.

Conclusion

Thus, Best First Search was successfully implemented using Prolog and heuristic-based node selection.